Notes 1 - Intro and GitHub

Math 3190 - Fundamentals of Data Science

Rick Brown

Southern Utah University

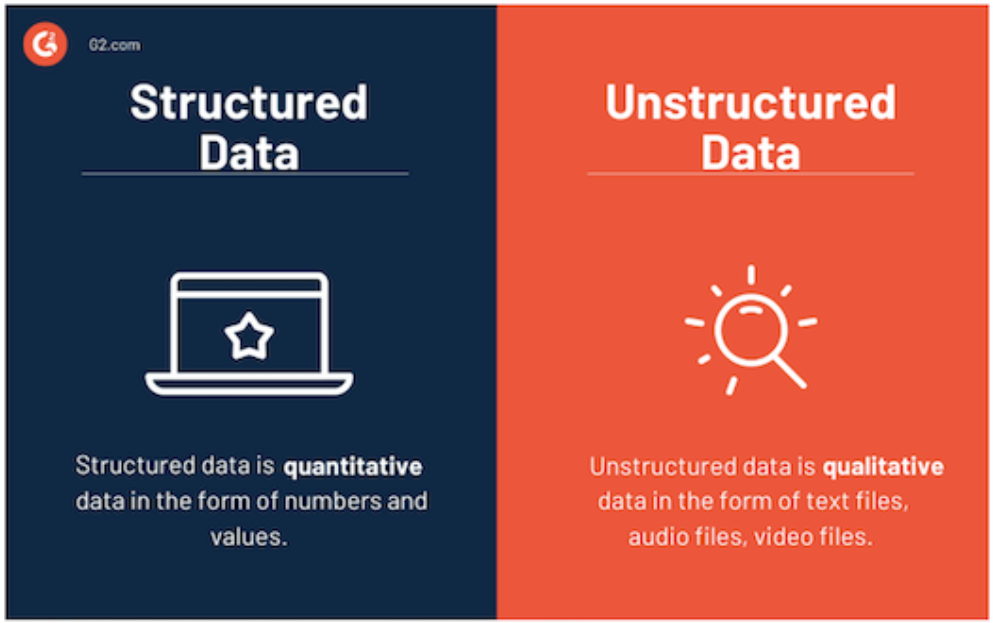
1. Introduction to Data Science
2. Installation Details
3. Introduction to Positron
4. Introduction to Unix
5. Unix Organization and Commands
6. Git and GitHub

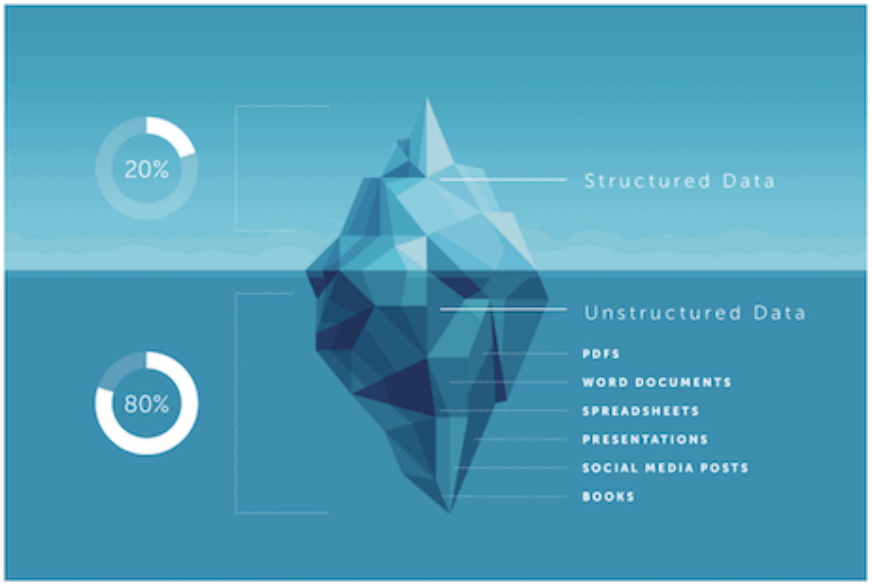
Section 1

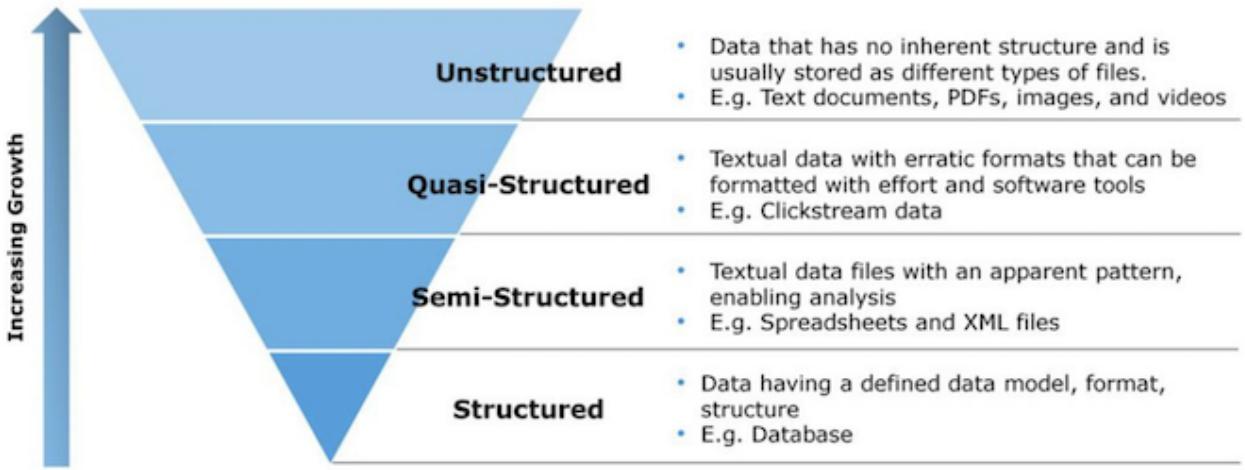
Introduction to Data Science

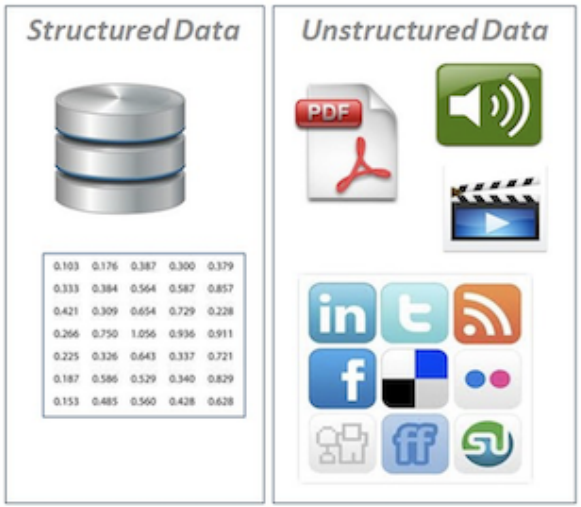
Big Data has fundamentally changed how we look at science and business. Along with advances in analytic methods, they are providing unparalleled insights into our physical world and society

Structured vs. Unstructured Data

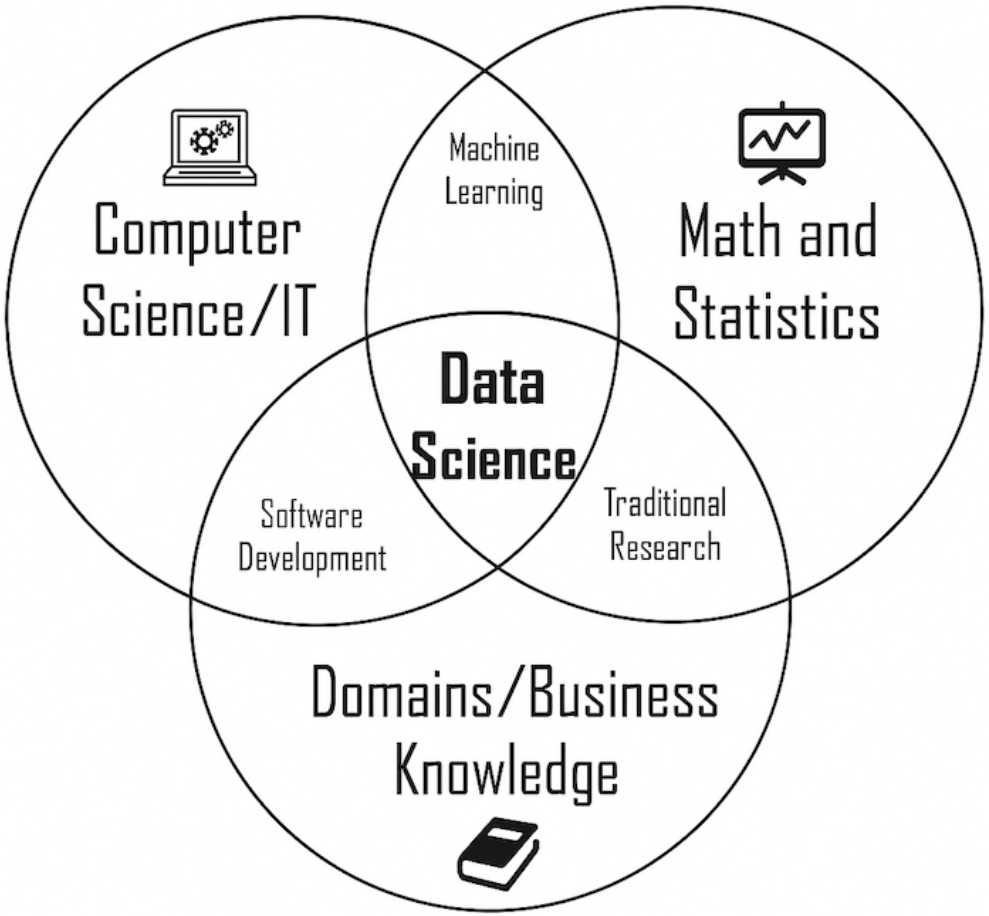






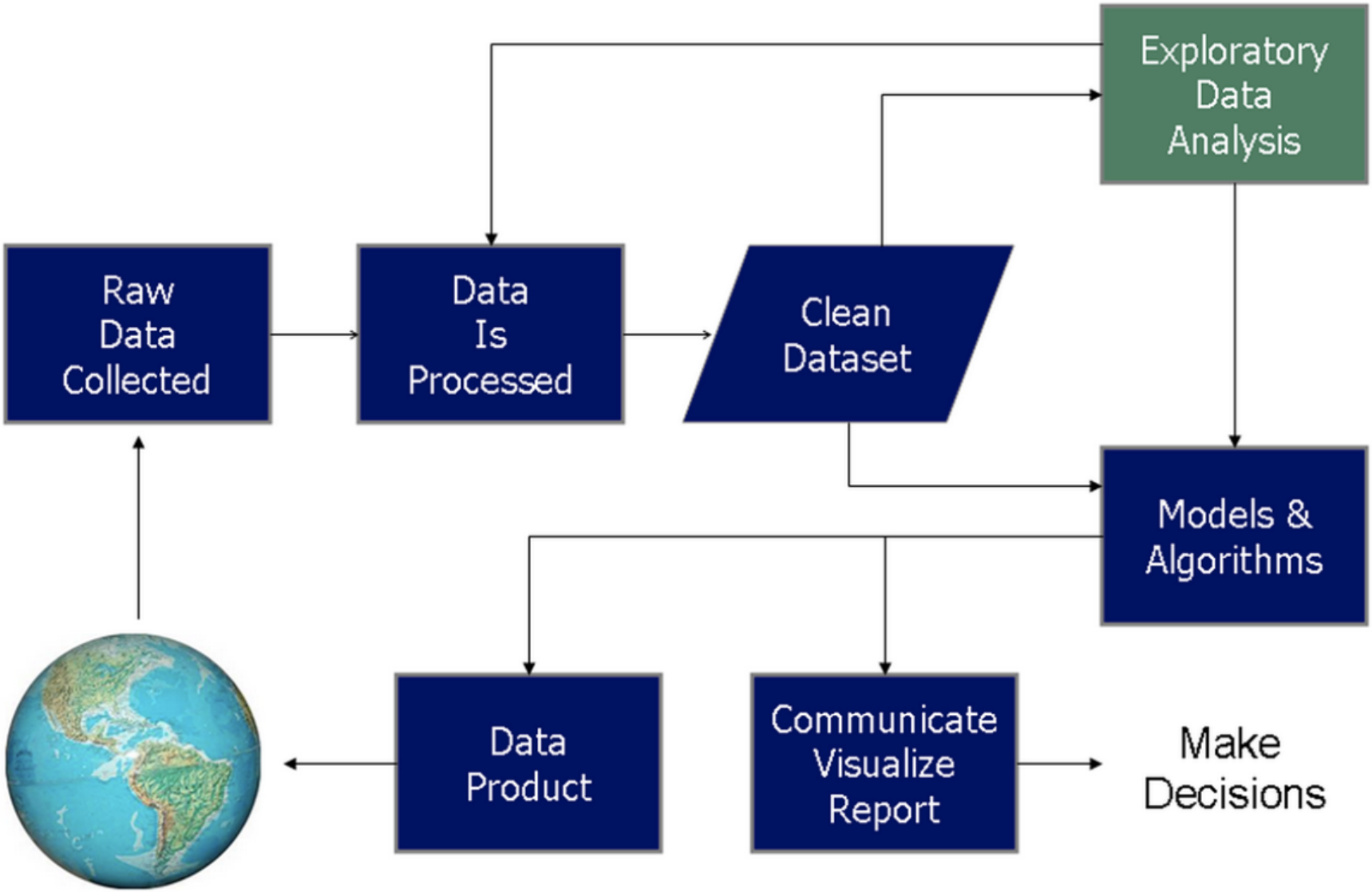


Data Science Revolution



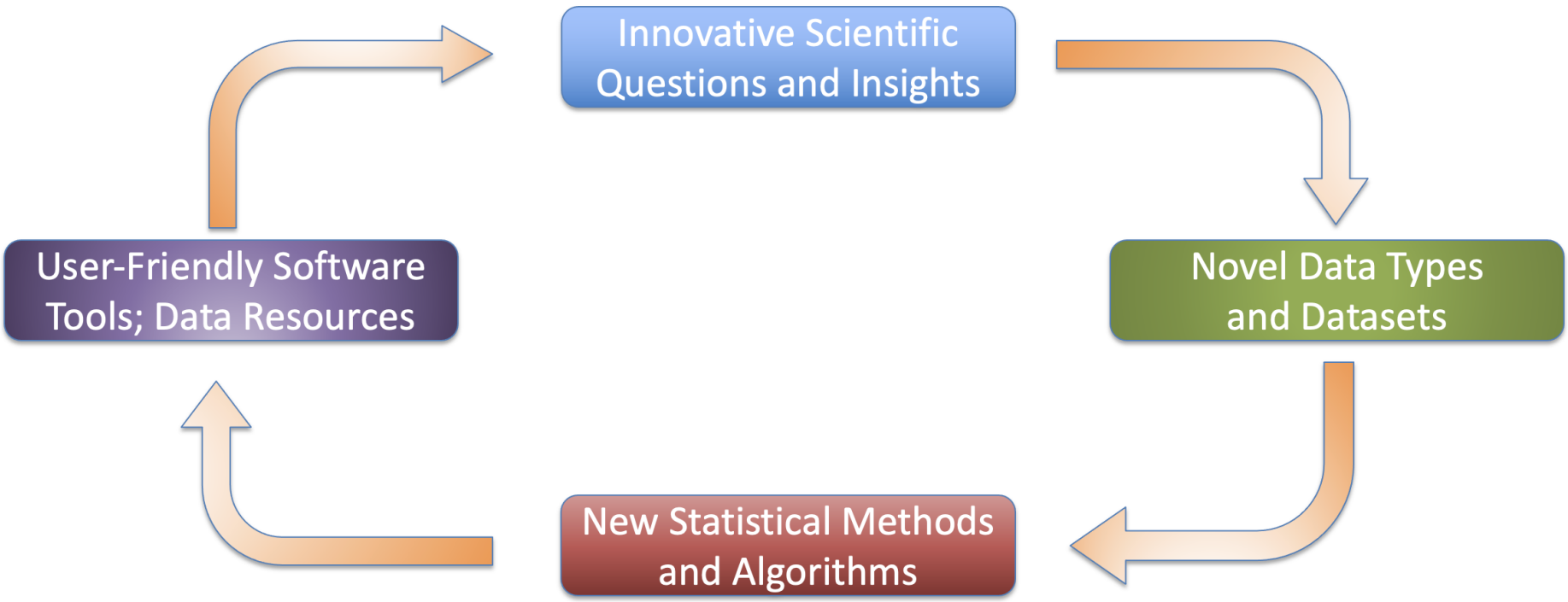
- Few have all the skills
- Flexibility in area (business, strategy, health care) and conditions
- Data science makes companies and data better!

Data Science Process



Scientific Cycle for Data Science

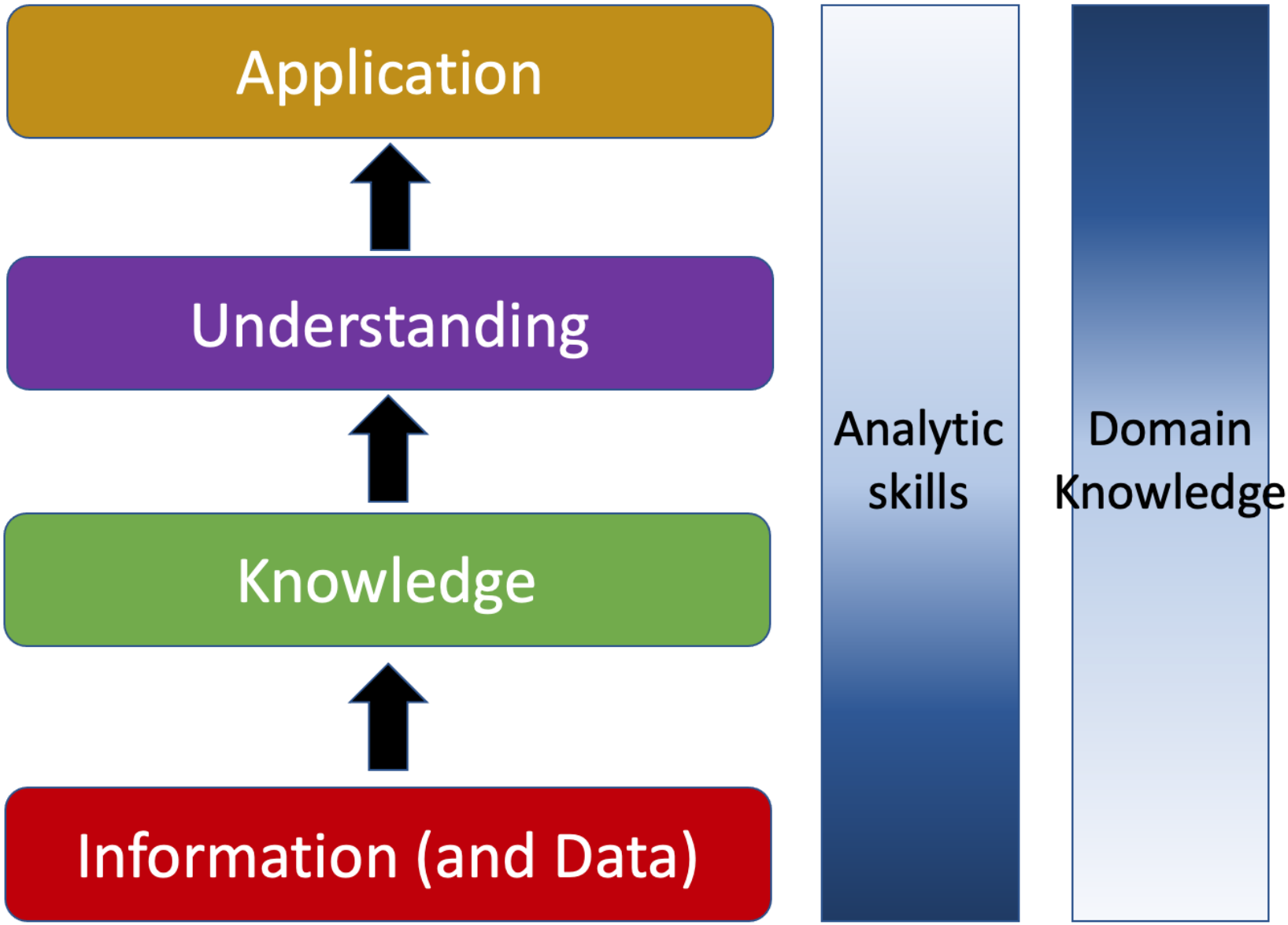
Common Approach to Science:



Domain Knowledge

Domain knowledge is knowledge of a specific, specialized discipline or field, in contrast to general (or domain-independent) knowledge. For example, in describing a software engineer may have general knowledge of computer programming as well as domain knowledge about developing programs for a particular industry. People with domain knowledge are often regarded as specialists or experts in their field. (Wikipedia!)

Analytixcs Hierarchy



Section 2

Installation Details

Important installations

You will need to install the following:

Mac Users

- R and R Studio or Positron (a new data science IDE!)
- Know how to access a terminal (RStudio, Positron or Terminal)
- Git (type `git --version` in the terminal)

Windows Users:

- R and Positron
- A terminal app (Git Bash [recommended], MobaXterm, Putty)
- Git for Windows

Important Installations

For installing **R** and RStudio: https://cloud.r-project.org and https://posit.co/download/rstudio-desktop

Install Positron here (optional): https://positron.posit.co/download.html

Install Git here: https://git-scm.com/install and if on Windows, that will also install Git Bash.

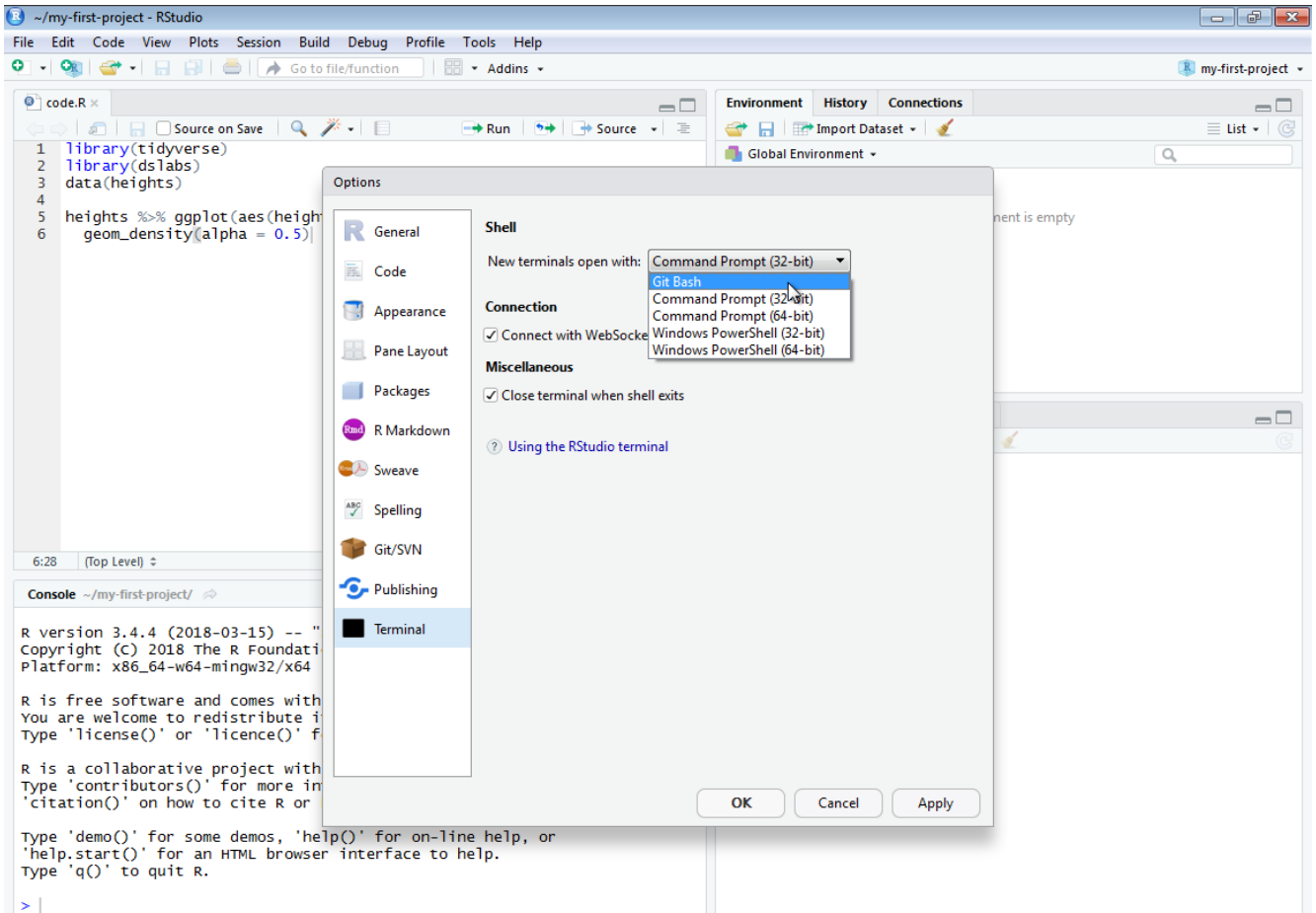
Note: When installing on Windows, make sure to chose the option to call the default branch "main" so it aligns better with GitHub. If you forget to do this, you can always change the default branch by typing

```
git config --global init.defaultBranch main
```

in the Git Bash terminal after installation.

For Windows: link Git Bash and RStudio

We can access the terminal either through RStudio or by opening Git Bash directly. For RStudio, set Git Bash as the default Unix shell: go to Tools then Global Options. . . , then select Terminal, then select Git Bash. It might have this already selected by default.



Section 3

Introduction to Positron

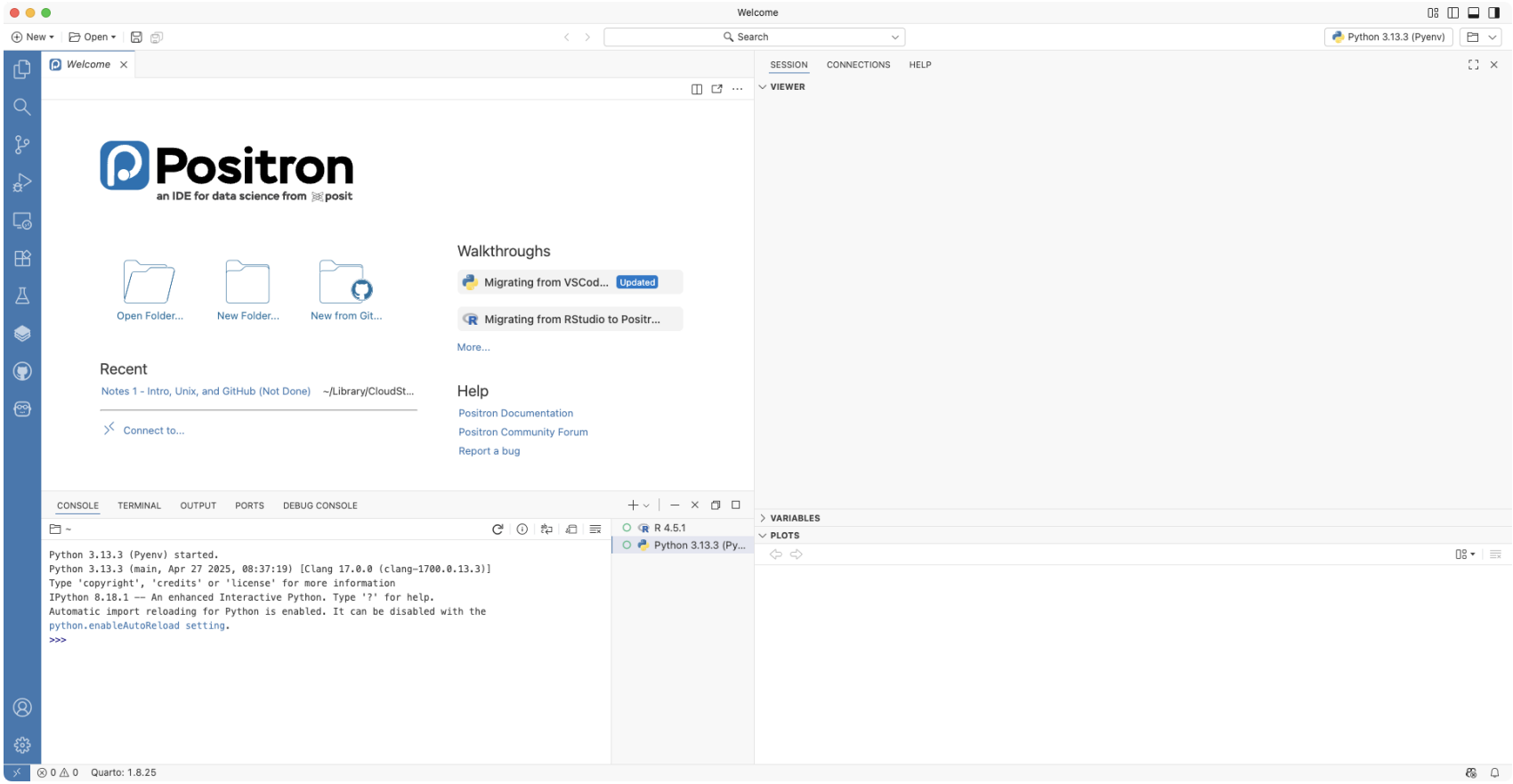
Positron

Positron is a next-generation IDE (Integrated Development Environment) that is designed specifically for data science. It is like a mix between VS Code and RStudio and runs both **R** and Python natively.

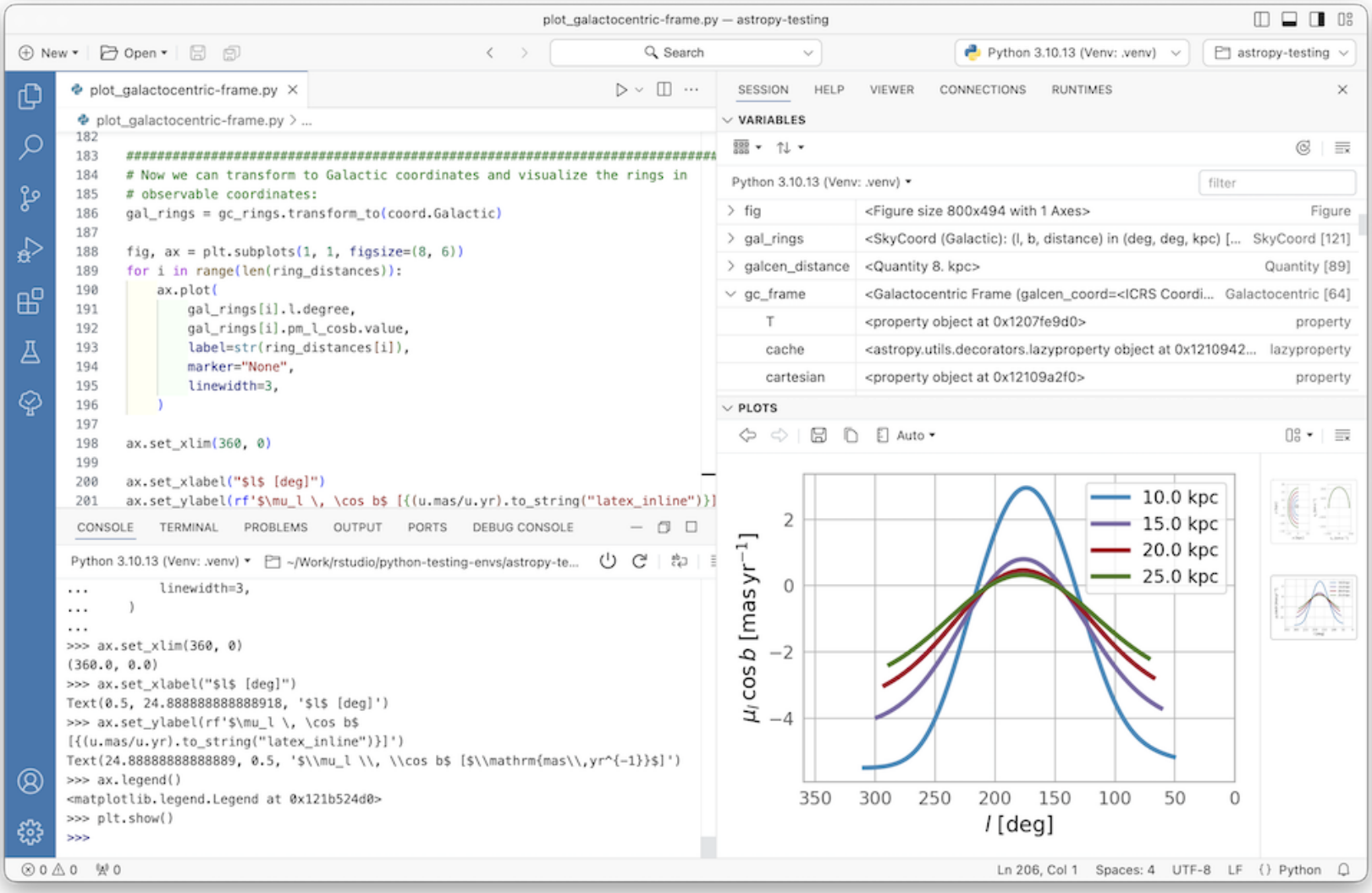
It is developed by Posit, which also created RStudio.

In this class, you can use RStudio or Positron. If you consider yourself primarily a Python programmer, then Positron may be a better fit for you than RStudio. However, I will give instructions is subsequety notes in RStudio, not Positron.

Positron First Impressions



Positron Typical Workflow



Getting Started with Positron

When you first open Positron, it will want you to update the extensions. Click on the Extensions pane on the left side (the one with 4 squares and a red number on it). Then choose to update all extensions.

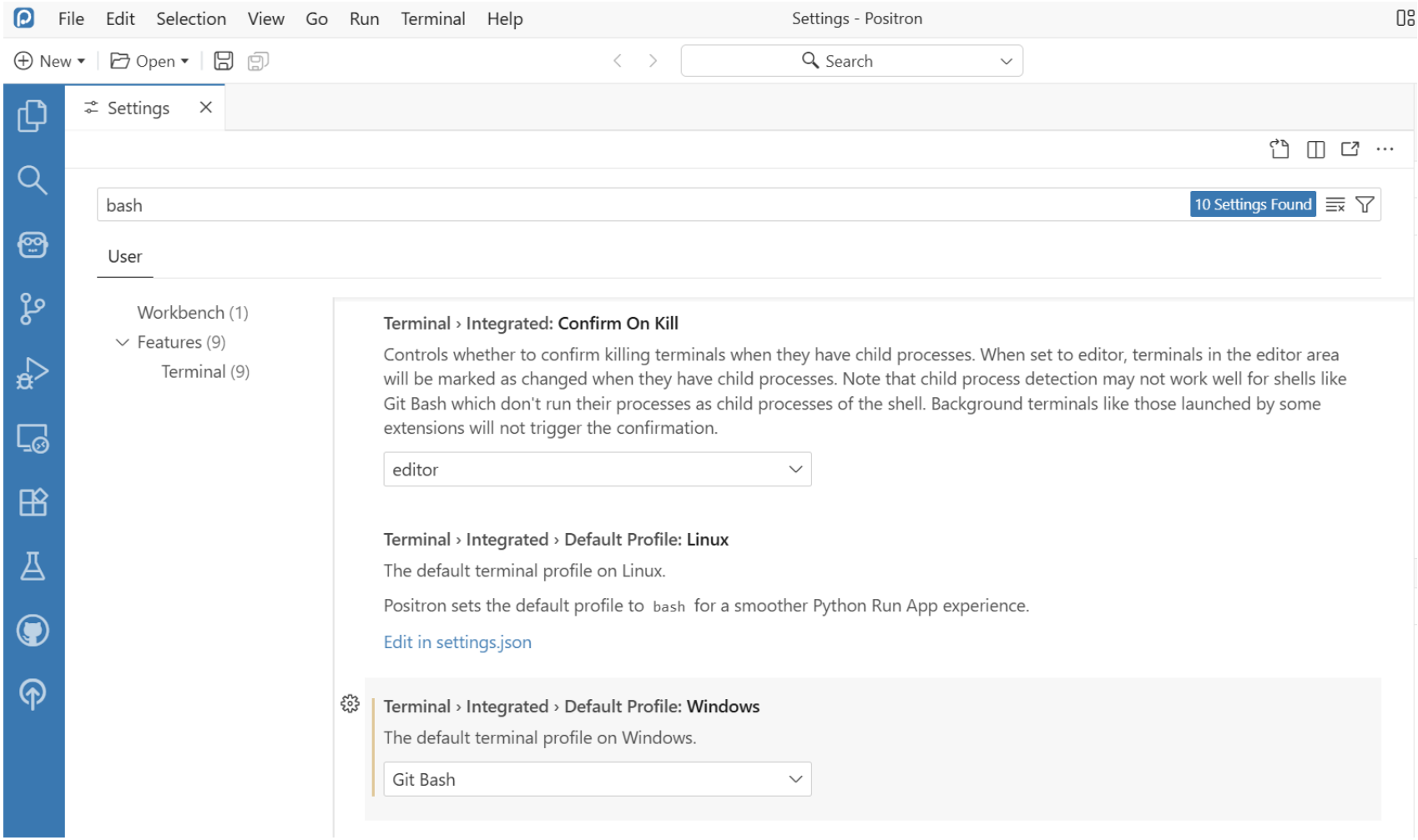
We will also need to make sure it connects to **R** (and Python if you have that installed). Click on "Start Session" in the top right. You should be able to click on the **R** version you have installed on your computer. If you have Python installed, you an easily swtich back and forth between **R** and Python.

For Windows: link Git Bash and Positron

We can access the terminal either through Positron or by opening Git Bash directly. For Positron, set Git Bash as the default Unix shell:

- Go to settings in the lower left.
- Search for Bash
- Change the Terminal > Integrated > Default Profile: Windows to Git Bash. (See image on next slide)
- Type Shift-Control-P and search for "Terminal: Create New Terminal".
- Git Bash terminal should now appear in the TERMINAL tab in Positron.

For Windows: link Git Bash and Positron



Section 4

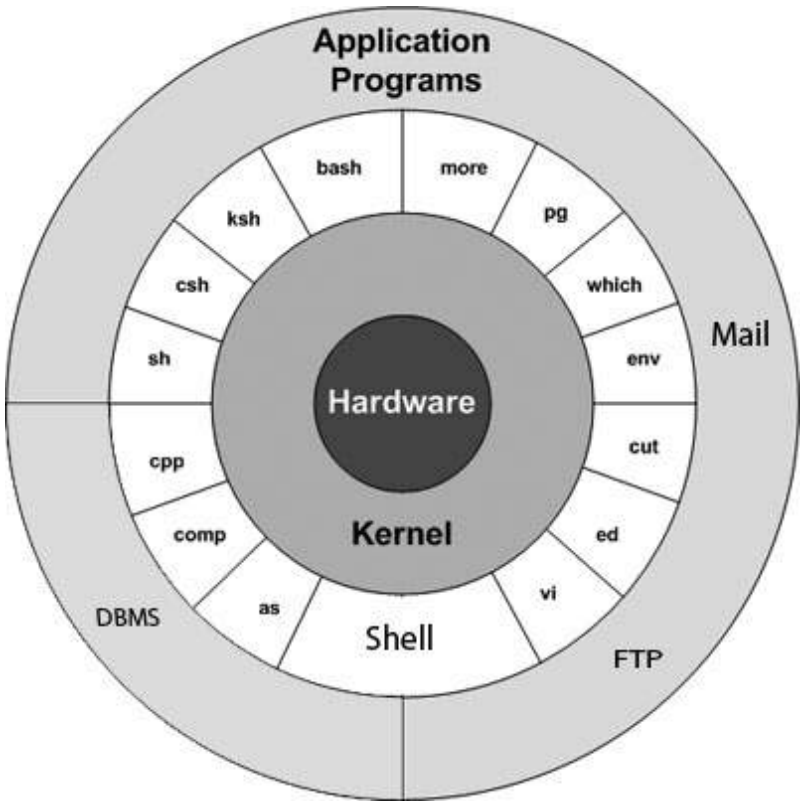
Introduction to Unix

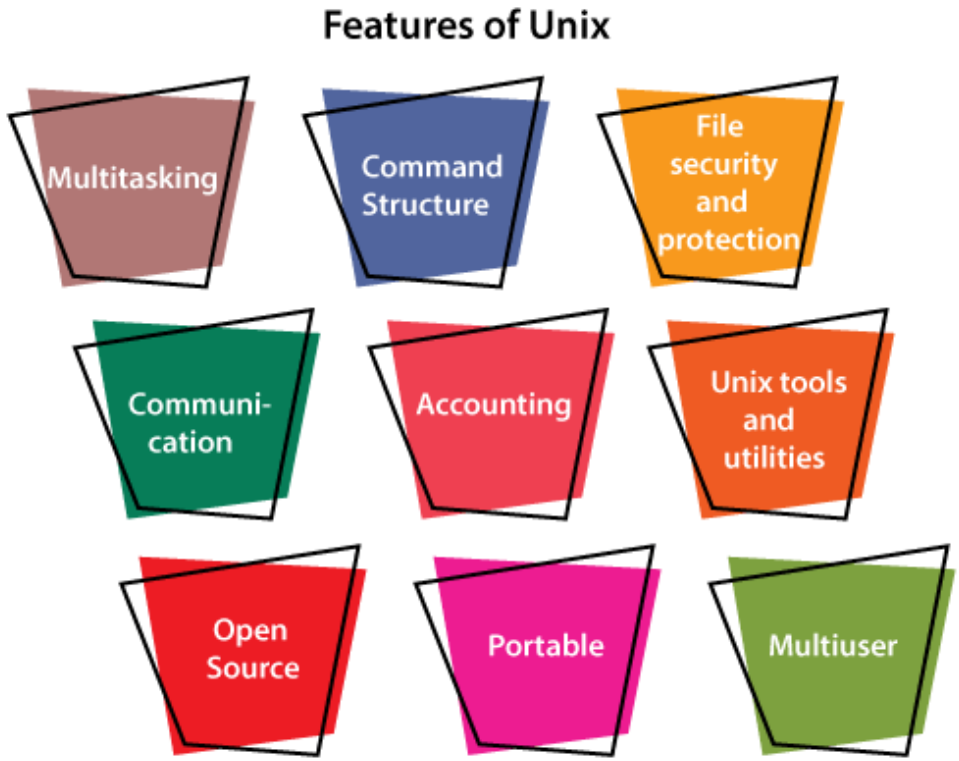
Unix Introduction

Unix is a family of multitasking, multiuser computer operating systems that derive from the original AT&T Unix. Originally developed in 1969 at Bell Labs. (Thanks Dennis Ritchie and Ken Thompson!)

Unix Introduction

The Unix operating system consists of many libraries and utilities along with the master control program, the kernel.

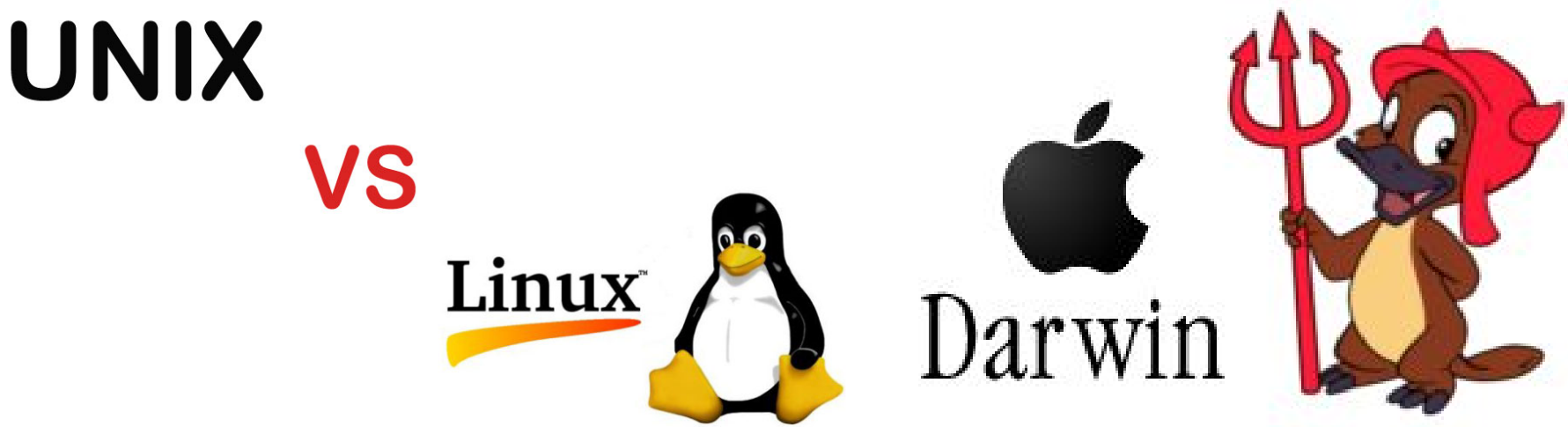




History of Operating Systems

Most operating systems can be grouped into two lineages:

- Microsoft's Windows NT-based operating systems
- "Unix-like" operating systems (e.g., Linux, macOS, Android, iOS, Chrome OS, Orbis OS used on the PlayStation 4, firmware on your router)



Unix and Data Science

Unix is the operating system of choice in data science.

We will introduce you to the Unix way of thinking using an example: how to keep a data analysis project organized, and later how to do things more efficiently.

We will learn some of the most commonly used commands along the way.

Unix Resources

Here are some resources for more information:

- https://www.codecademy.com/learn/learn-the-command-line
- https://www.edx.org/course/introduction-linux-linuxfoundationx-lfs101x-1
- https://www.coursera.org/learn/unix

There are many reference books as well. Here are some particularly clear, succinct, and complete examples:

- https://www.quora.com/Which-are-the-best-Unix-Linux-reference-books
- https://gumroad.com/l/bite-size-linux
- https://jvns.ca/blog/2018/08/05/new-zine--bite-size-command-line

Naming Conventions

Before you start, pick a name convention that you will use to systematically name your files and directories. The Smithsonian Data Management Best Practices has "five precepts of file naming and organization":

- Have a distinctive, human-readable name that gives an indication of the content.
- Follow a consistent pattern that is machine-friendly.
- Organize files into directories (when necessary) that follow a consistent pattern.
- Avoid repetition of semantic elements among file and directory names.
- Have a file extension that matches the file format (no changing extensions!)

The Tidyverse Style Guide[1] is highly recommended!

[1] https://style.tidyverse.org/

Section 5

Unix Organization and Commands

Basic Unix Commands

In addition to clicking, dragging, and dropping files and folders, we will be typing Unix commands into the terminal. (Similar typing commands into the **R** console!)

You will need access to a terminal (see installation instructions in Installation Details section). Once you have a terminal open, you can start typing into the *command line*, usually after a $ or % symbol. Once you hit enter, Unix will try to execute this command. For example:

```
echo "hello world"
```

Basic Unix Commands

The command `echo` is similar to `cat()` or `print()` in **R**. Executing this line should print out `hello world`, then return back to the command line.

Notice that you can't use the mouse to move around in the terminal like you ordinarily do. You have to use the keyboard or hold the Option key and then click on a Mac. To go back to a previous command, you can use the up arrow.

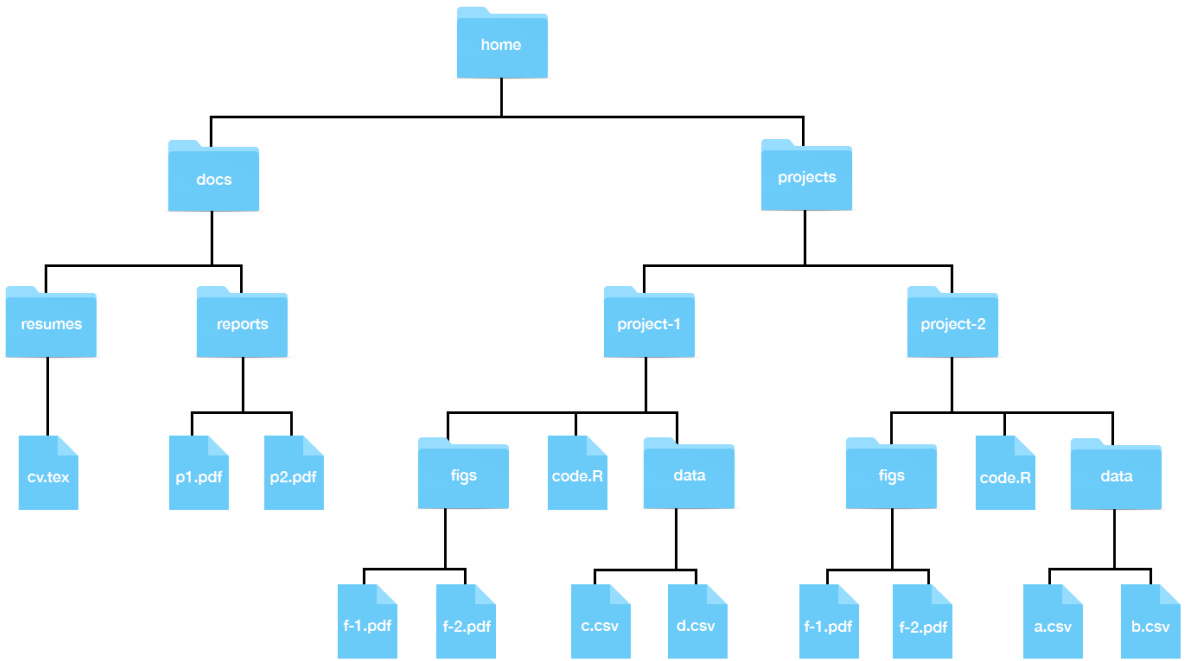
The Filesystem

We refer to all the files, folders, and programs on your computer as the **filesystem**. Keep in mind that folders and programs are also files (for later discussion). We will focus on files and folders for now and discuss programs, or **executables**, in a later section.

The first concept to understand is how your filesystem is organized; a series of nested folders, each containing files, folders, and executables.

Directories and Subdirectories

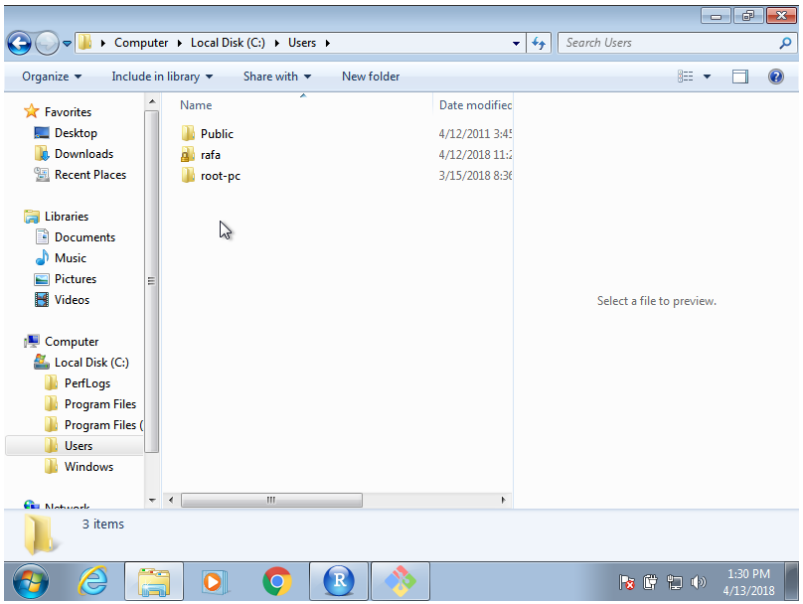
Here is a representation of the structure we are describing:

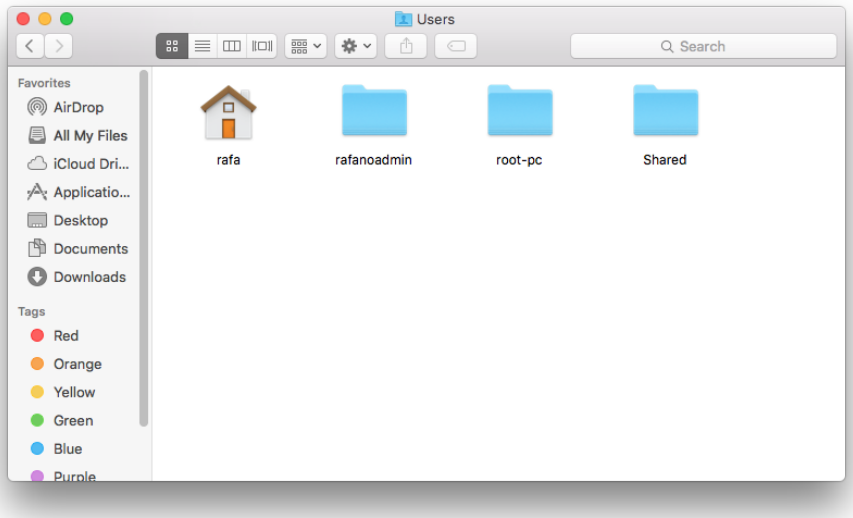


In Unix, we refer to folders as **directories**. Directories inside other directories referred to as **subdirectories**.

The Home Directory

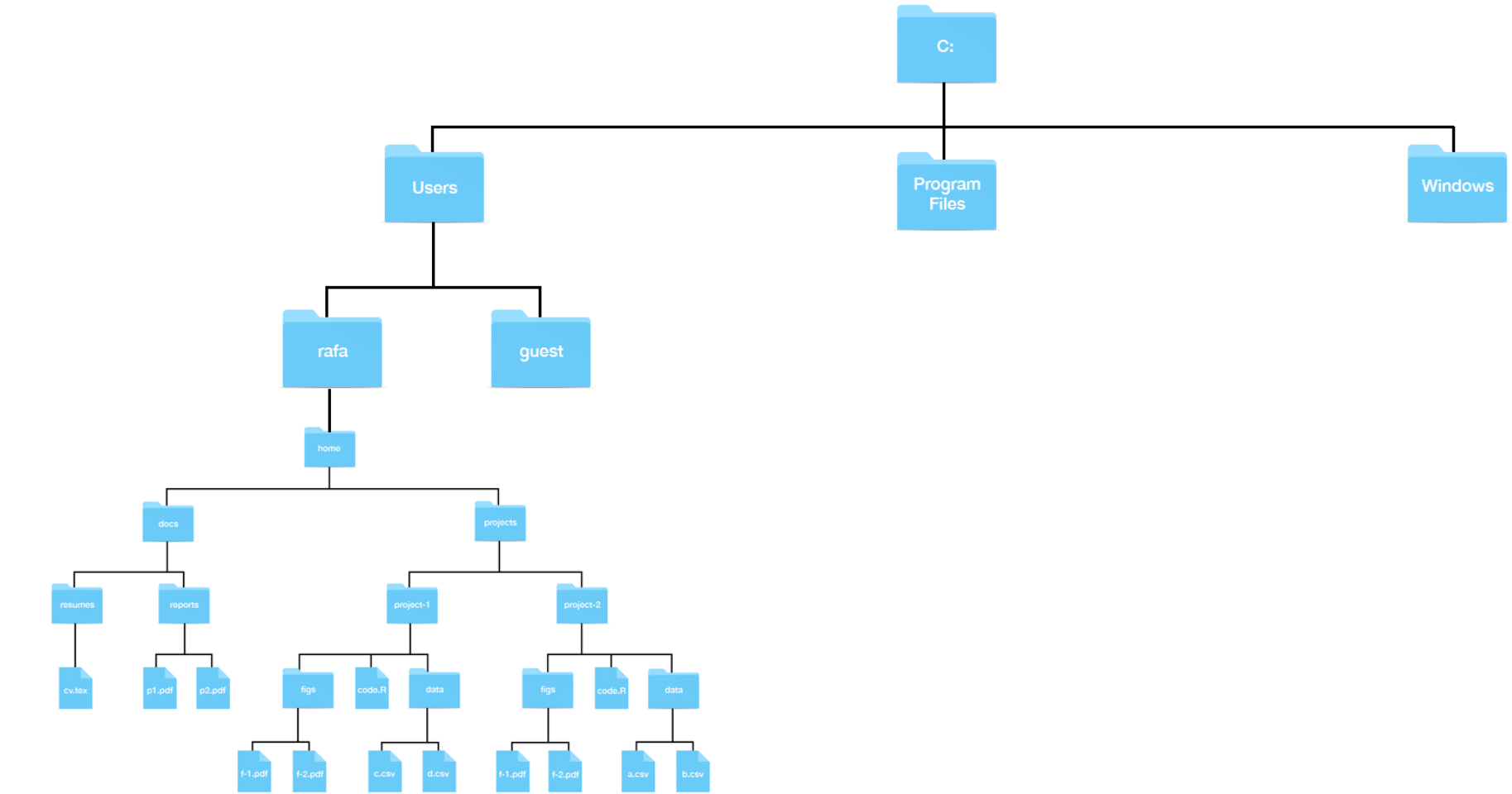
The **home directory** is where your stuff is kept, as opposed to the system files, which are kept elsewhere. The name of your home directory is likely the same as your username. For example:





Windows Filesystem Structure

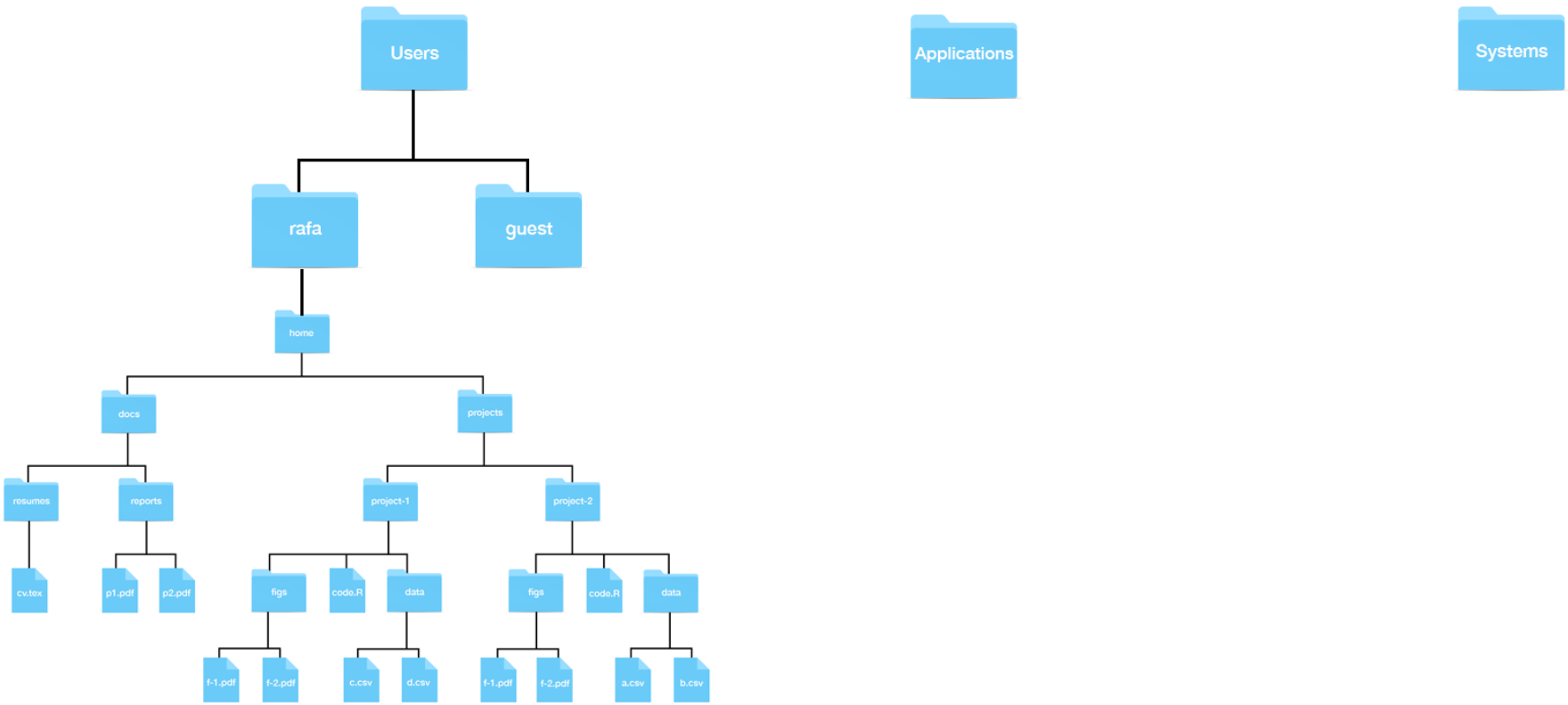
The Windows filesystem structure[2] looks like this:



[2]**Windows Users:** The typical **R** installation will make your `Documents` directory your home directory in **R**. This will likely be different from your home directory in Git Bash.

Mac Filesystem Structure

The Mac filesystem structure looks like this:



Working Directory

In Unix, the concept of a current location is indispensable. We refer to this as the **working directory**. Each terminal window you have open has a working directory.

How do we know our working directory? We use the Unix command: `pwd`, which stands for *print working directory*:

```
pwd
```

Paths

The string returned by `pwd` is the **full path** of the working directory. Every directory has a full path. We can also reference **relative paths**, which tell us how to get to a directory from the working directory.

In Unix, the shorthand `~` is a nickname for your home directory. So, for example, if `docs` is a directory in your home directory, the full path can be written `~/docs`.

Some Basic Unix Commands

To see the content of your home directory, open a terminal and type:

```
ls
```

You can also add **options** (- and --), **arguments**, and **wild cards** ('*') to change function behavior:

```
ls -l
ls -a
ls -t
ls -r
ls -lart
ls -l *.txt
```

Some Basic Unix Commands

To create and remove directories, use the `mkdir` and `rmdir` functions, for example (use `ls` in between to see the directory come and go):

```
mkdir projects
mkdir junk
rmdir projects
```

Note: `rmdir` does not work if the directory is not empty.
Pro tip 1: The up arrow retrieves your previous commands.
Pro tip 2: You can auto-complete by hitting tab.
Pro tip 3: Type "Control-C" to clear a command.

Some Basic Unix Commands

To change the working directory use the `cd` command, which stands for *change directory*. For example:

```
cd projects
```

To check that it worked, use `pwd`. Now try:

```
cd .
cd ..
cd ../..
cd /
cd ~
cd ~/projects
cd ../junk
```

Some Basic Unix Commands

In Unix, we move files from one directory using the `mv` command:

```
mv path-to-file path-to-destination-directory
```

Warning: `mv` will not ask "are you sure?" if your move results in overwriting a file. Some `mv` examples:

```
mkdir ~/junk ~/projects/resumes
mv ~/projects/resumes/cv.tex ../../
mv ~/cv.tex ~/mycv.tex
mv ~/projects/mycv.tex junk/
mv ~/junk/mycv.tex ../projects/resumes/
```

Some Basic Unix Commands

The command `cp` behaves similar to `mv` except instead of moving, we copy the file. We first list the file we want to copy and then we list where we want to copy it and what its new name should be.

```
cp ~/projects/resumes/mycv.tex ~/mycv.tex
```

So in all the `mv` examples in the prior slide, you can switch `mv` to `cp` and they will copy instead of move. However to copy entire directories, add the recursive (`-r`) option:

```
cp -r ~/projects/resumes ~/junk/
```

Some Basic Unix Commands

In Unix, we can create a new blank file using the `touch` command.

```
touch filename
```

If that command is run again, then the file does not change except the modified date gets updated.

We remove files (and directories) with the `rm` command.

```
rm filename
```

Warning: Unlike throwing files into the trash or recycle bin, `rm` is permanent. Be careful! Note the following:

```
rm mycv.tex
rm junk/*.tex
rm -r projects junk
```

Some Basic Unix Commands

Unix uses an extreme version of abbreviations, which makes it hard to guess how to call commands. However, Unix includes complete help files or **man pages** (man is short for manual). In most systems, you can type `man` followed by the command name to get help. For example:

```
$ man ls
```

This command is not available in some of compact implementations of Unix (e.g., Git Bash). Alternatively, we can type the command followed by `-help`:

```
ls --help
```

Pro tip 4: Type `q` to quit out of a manual listing.

Advanced Unix functions

Most Unix implementations include a large number of powerful tools and utilities. It will take time to become comfortable with Unix, but as you struggle (if you do), you will find yourself learning just by looking up solutions on the internet.

Section 6

Git and GitHub

Git and GitHub Introduction

Many software developers and data scientists rely on Git and GitHub everyday.

There are three main reasons to use Git and GitHub.

- Sharing: GitHub allows us to easily share code with or without the advanced version control functionality.
- Collaborating: Multiple people make changes to code and keep versions synched. GitHub also has a special utility, called a **pull request**, that can be used by anybody to suggest changes to your code.
- Version control: The version control capabilities of Git permit us to keep track of changes, revert back to previous versions, and create **branches** in to test out ideas, then decide if we **merge** with the original.

See the Installation Details section for installing Git on your system.

GitHub Accounts

After installing Git, you need to create a GitHub account. To do this, go to https://github.com where you will see a link to sign up in the top right corner.

Pick a name carefully! Choose something short, somehow related to your name, and professional. Remember that you will use this to share code with others and you might be sharing this with potential collaborators or future employers!

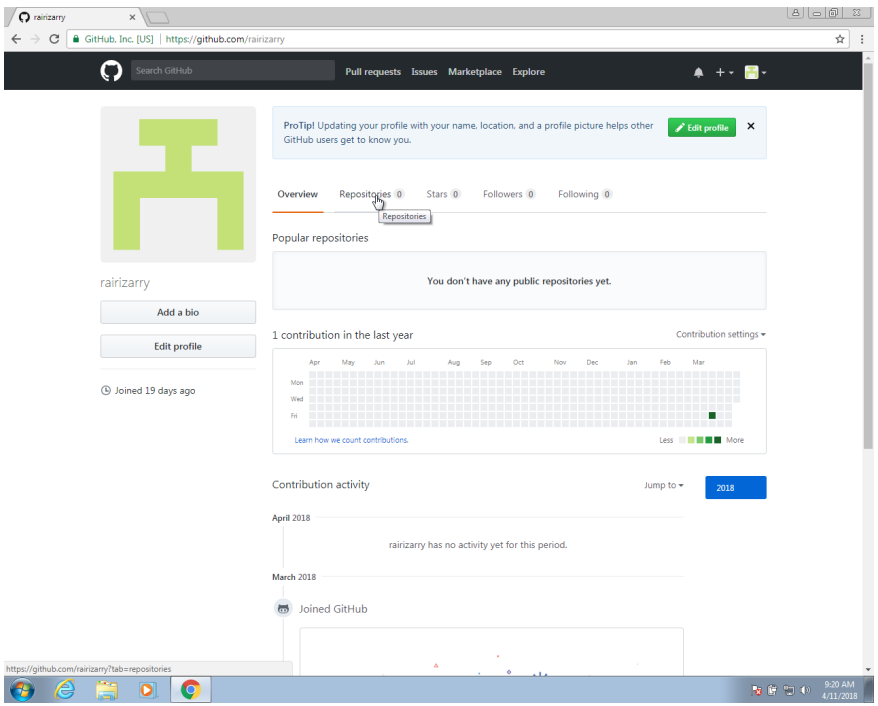
GitHub Repositories

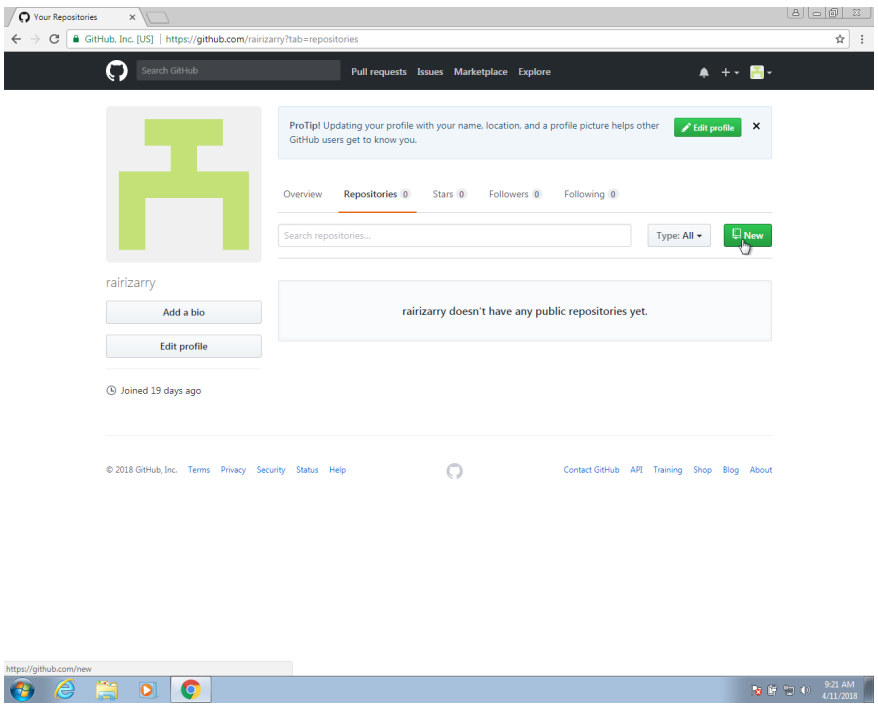
A GitHub **repository** (repo for short) is like a folder on your computer and allows you to have two copies of your code: one on your computer and one on GitHub. If you add collaborators, then each will have a copy on their computer.

The GitHub copy is usually considered the **main** copy (previously called the **master** and still referred to as master in many tutorials and pages). Git will help you keep all the different copies synced.

GitHub Repositories

The first step is to initialize the repository on GitHub. You will have a page on GitHub with the URL: http://github.com/username. On your account, you can click on *Repositories* and then click on *New* to create a new repo:

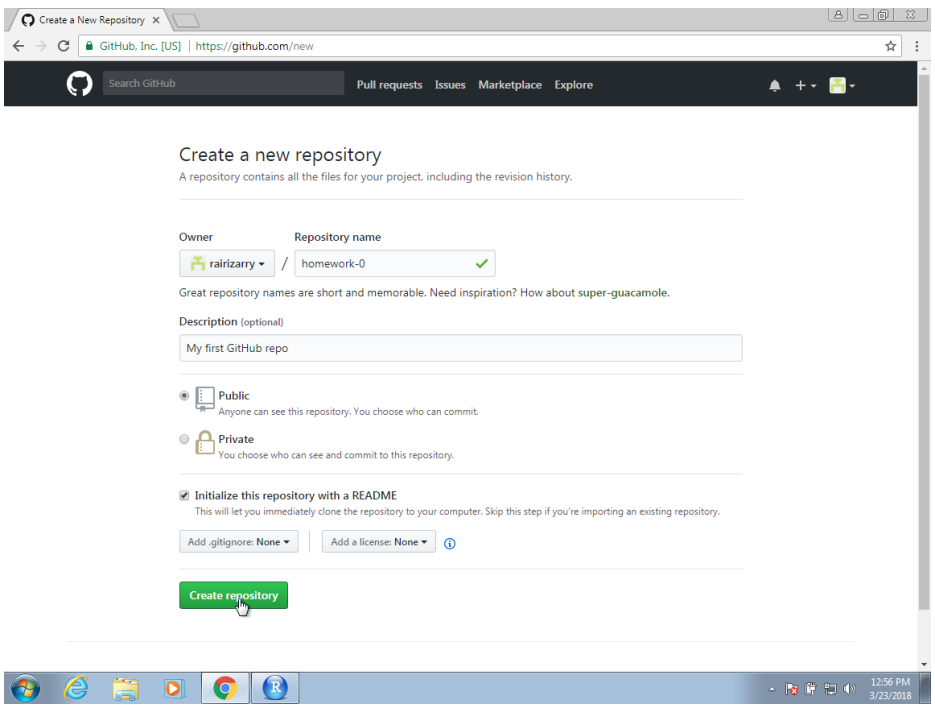


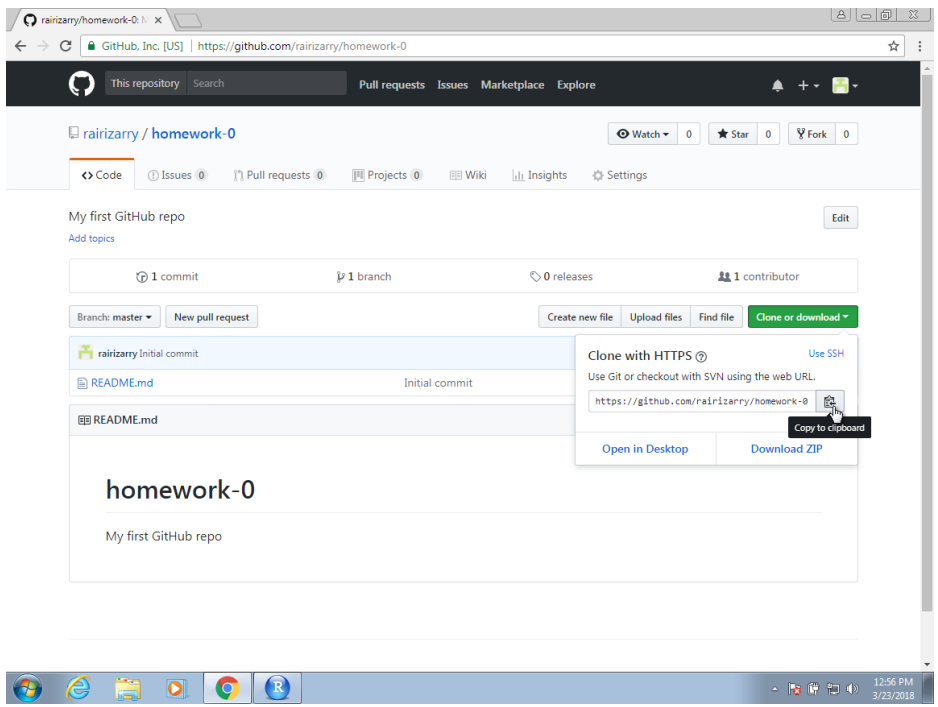


GitHub Repositories

Choose a good descriptive name. In your first lab, you will create one called "Math_3190_Assignments". You don't need to add a `README.md`, a `.gitignore` or choose a license at this time.

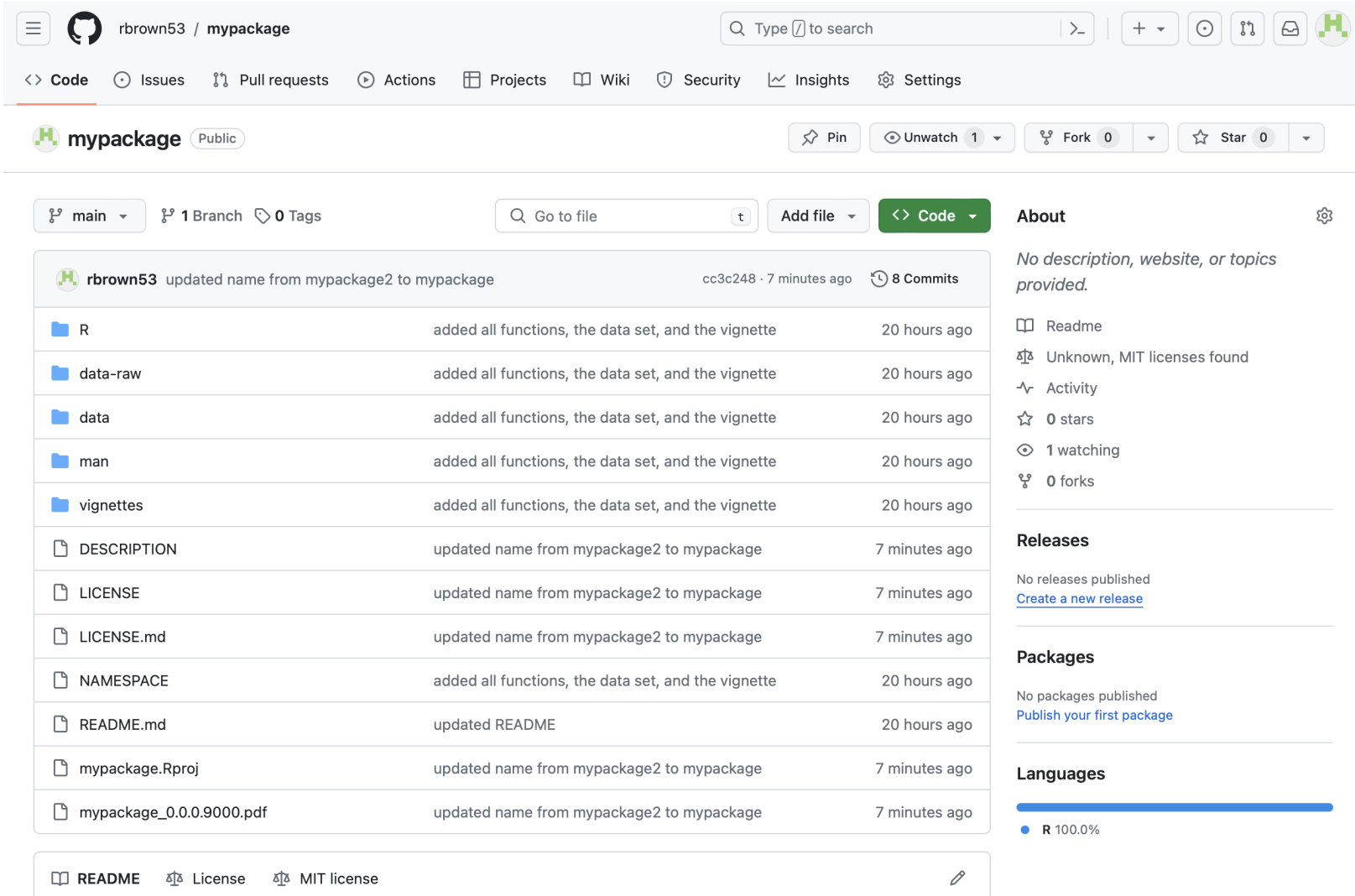
You will then be able to **clone** it on your computer using the terminal. Copy the link to connect to this repo for the next step.





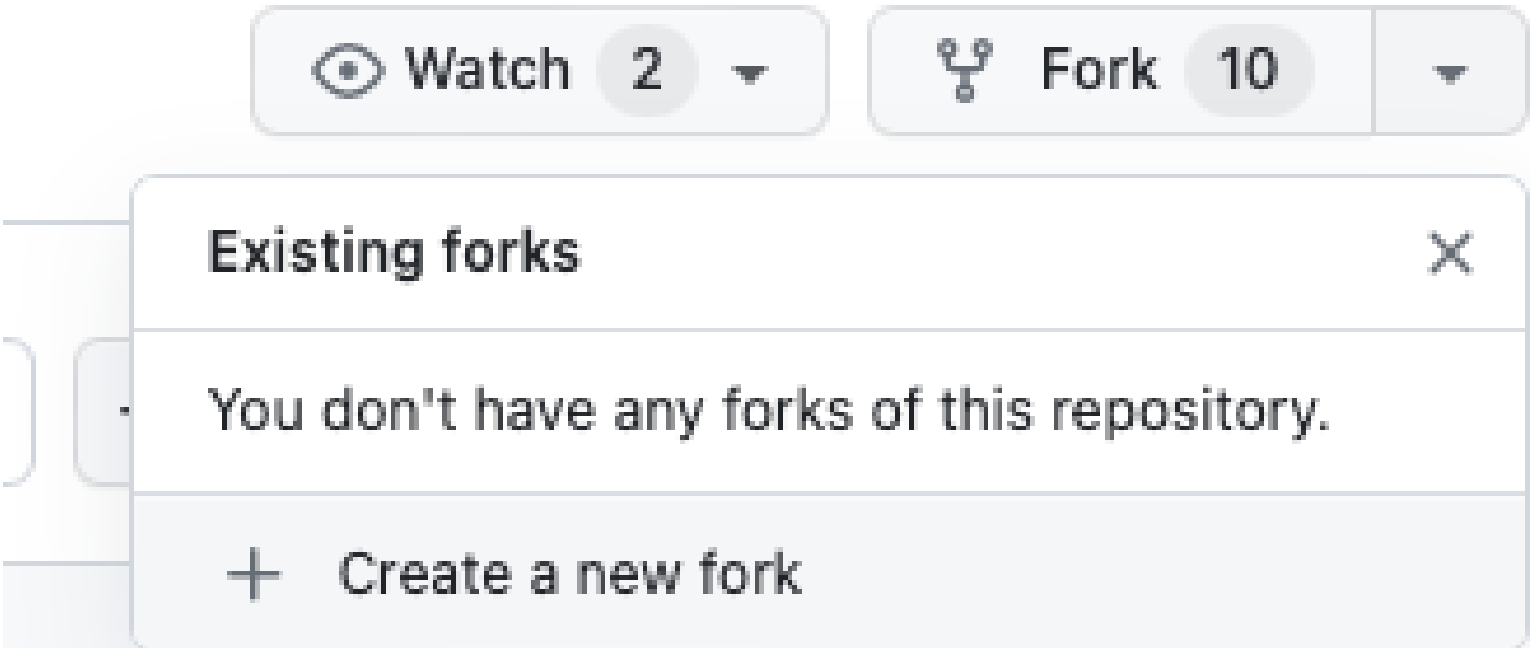
Forking

GitHub also allows you to **fork** others' repos. Go to https://github.com/rbrown53/mypackage



Forking

Click *Fork* in the top right and this will create a fork of the repository in your GitHub account. This will allow you to alter the code of other people (assuming you are allowed to do so by their license).



Git Setup - Tokens

Instead of using passwords, to connect local folders with GitHub repositories, we need to generate a GitHub **token**. Here are steps for obtaining one:

1. Go to your GitHub home page: https://github.com/username
2. Click on your account icon on the top right and go down to "Settings".
3. On the left side, at the bottom, click on "Developer Settings".
4. On the left, click on "Personal access tokens".
5. Click on Tokens (classic). We will be using the classic tokens, not the fine-grained tokens.
6. Click on "Generate new token" and the "Generate new token (classic)".
7. Select an expatriation time. 90 days is good.
8. Select "rep", "workflow", "gist", and "user" under scopes.
9. Click on "Generate token".

More info can be found here:
https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/managing-your-personal-access-tokens

Git Setup

Now we will work a bit in the Terminal. We need to let Git know who we are. This will make it clear who is making changes to files and directories. In a terminal window use the `git config` command:

```
git config --global user.name "My Name"
git config --global user.mail "my@email.com"
```

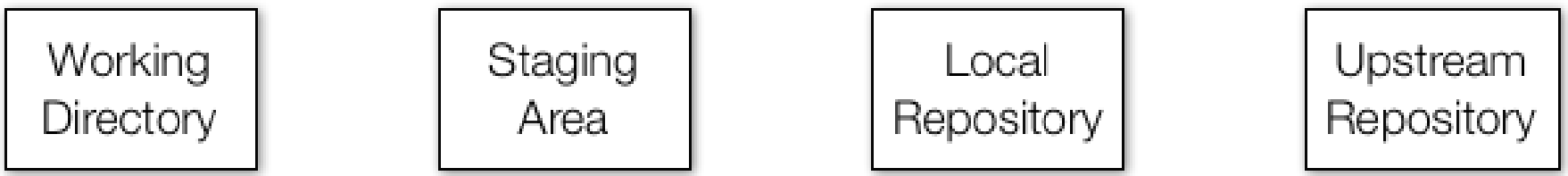
Git Setup

The main actions in Git are to:

1. **pull** changes from the remote GitHub repo.
2. **add** files to the staging area, or as we say in the Git lingo: **stage** files.
3. **commit** changes to the local repository.
4. **push** changes to the **remote** GitHub repo.

Git Setup

To effectively permit version control and collaboration in Git, files move across four different areas:

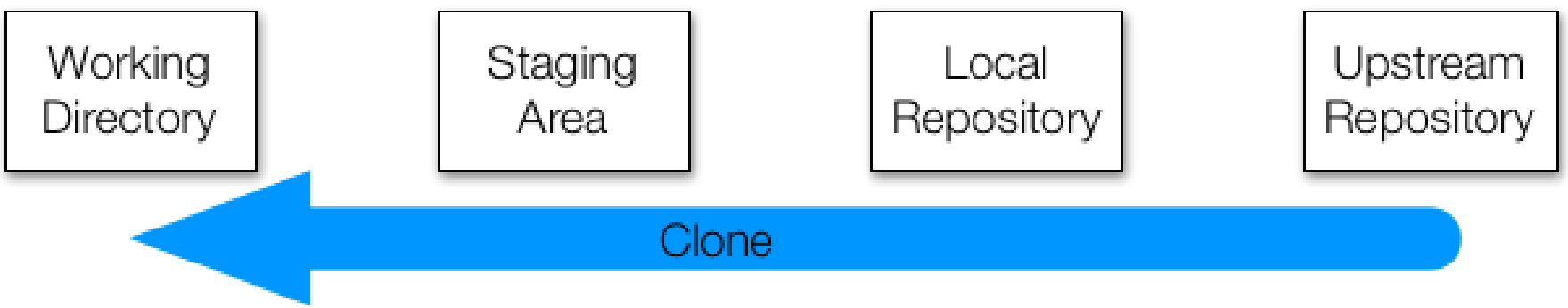


But how does it all get started? We can clone an existing repo or initialize one. We will explore cloning first.

Cloning Git Repositories

We will **clone** your existing my.package **upstream repository** to your local computer.

What does clone mean? We are going to actually copy the entire Git structure, files and directories to all stages: working directory, staging area, and local repository.



Cloning Git Repositories

Eventually, you will fork the mypackage repo from my GitHub. Then you will want to clone it to work on it on your local machine.

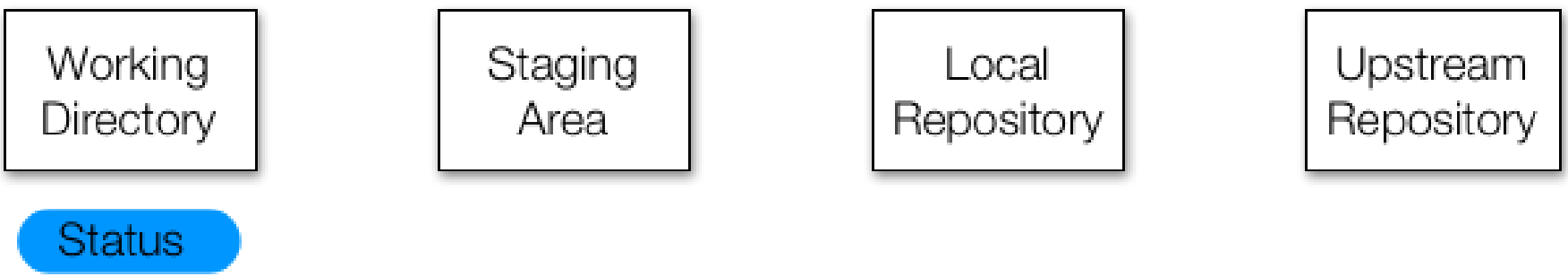
We can open a terminal and type:

```
cd ~
pwd
mkdir git-example
cd git-example
git clone https://github.com/yourusername/mypackage.git
cd mypackage
ls
```

Cloning Git Repositories

Note: the **working directory** is the same as your Unix working directory. When you edit files (e.g. in RStudio), you change the files in this directory. Git can tell you how these files relate to the upstream directory:

```
git status
```



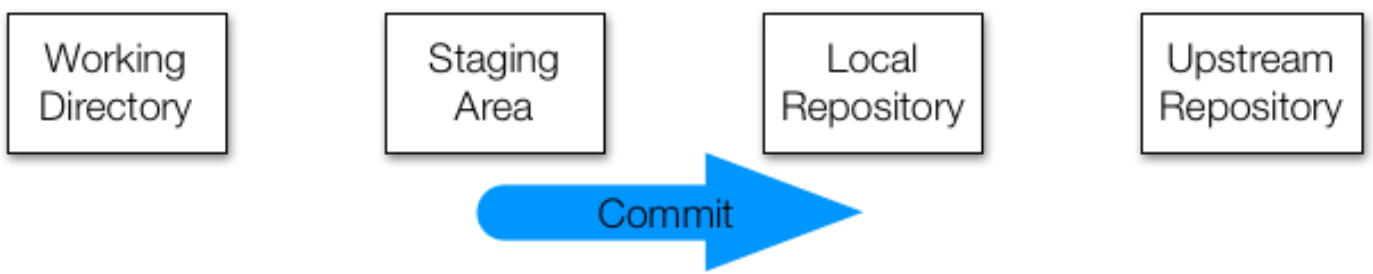
Now let's add some changes to the local repository. Open the `DESCRIPTION` file in the `mypackage` directory, and make a change.

Working with Git Repositories

Now lets **add** the changes to the staging area and **commit** the changes to the local Git directory:

```
git add DESCRIPTION
```


```
git commit -m "updated DESCRIPTION with 'hello' function"
```

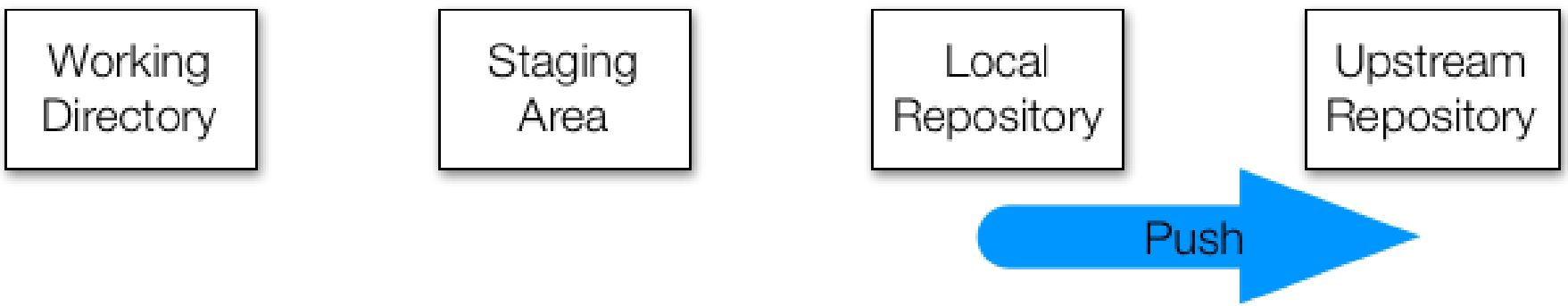


Working with Git Repositories

Now we can **push** the changes to the remote repo:

```
git push
```

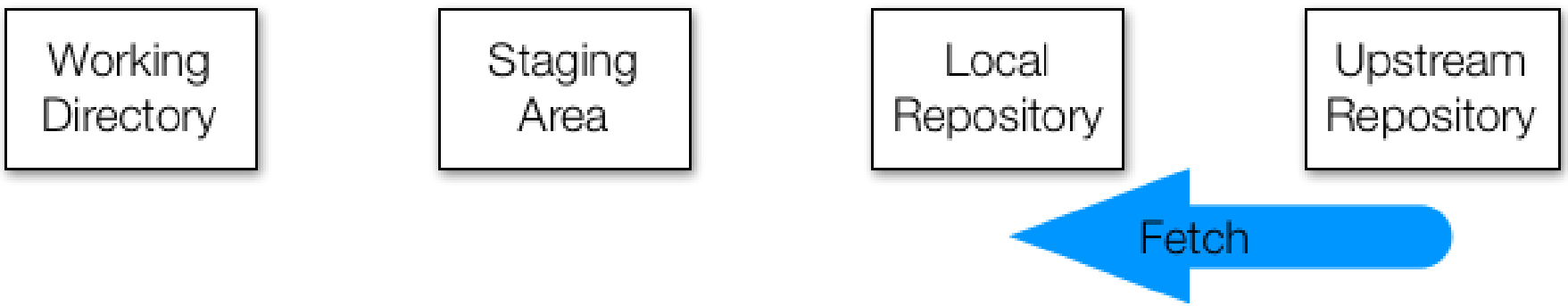
The system will ask for your username and password. The username is your GitHub username, but the password is your personal token that we generated earlier.



Working with Git Repositories

We can also **fetch** any changes on the remote repo (how do you think this is different from `clone`?):

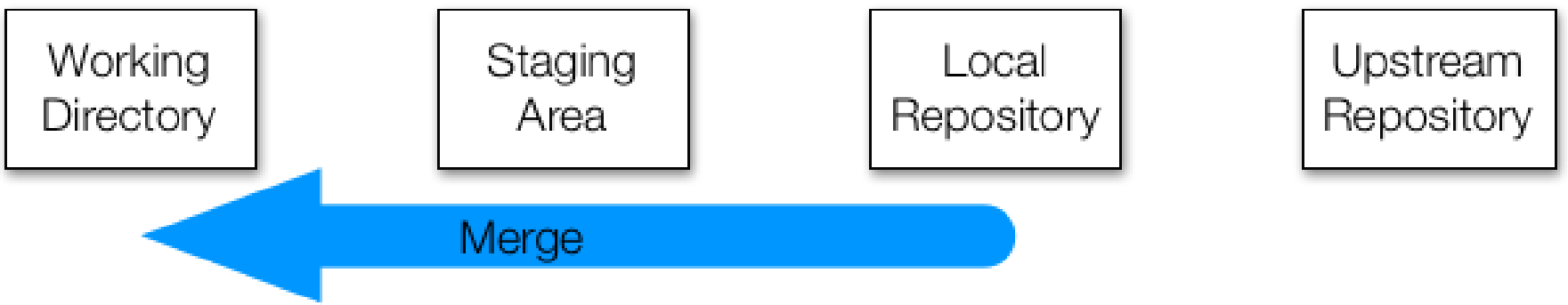
```
git fetch
```



Working with Git Repositories

And then we need to **merge** these changes to our staging and working areas:

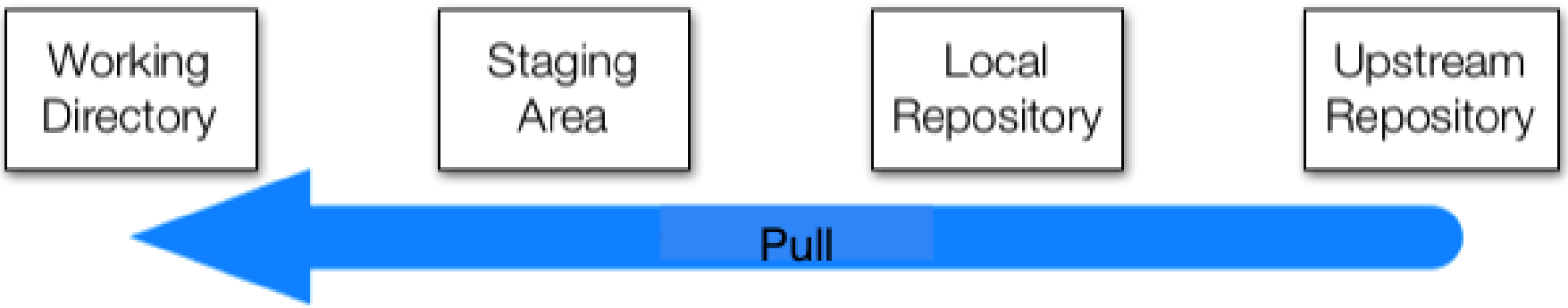
```
git merge
```



Working with Git Repositories

However, we often just want to change both with one command. For this, we use:

```
git pull
```



Important note: it is often a good idea to pull any changes when you start each day, so as to avoid **conflicts**.

Initializing a Git Directory

What if we already have a local directory and want to move it to a GitHub repository? See the following:

1. Create a new GitHub repository (e.g., Math_3190_Assignments)
2. **Initialize** the local repository

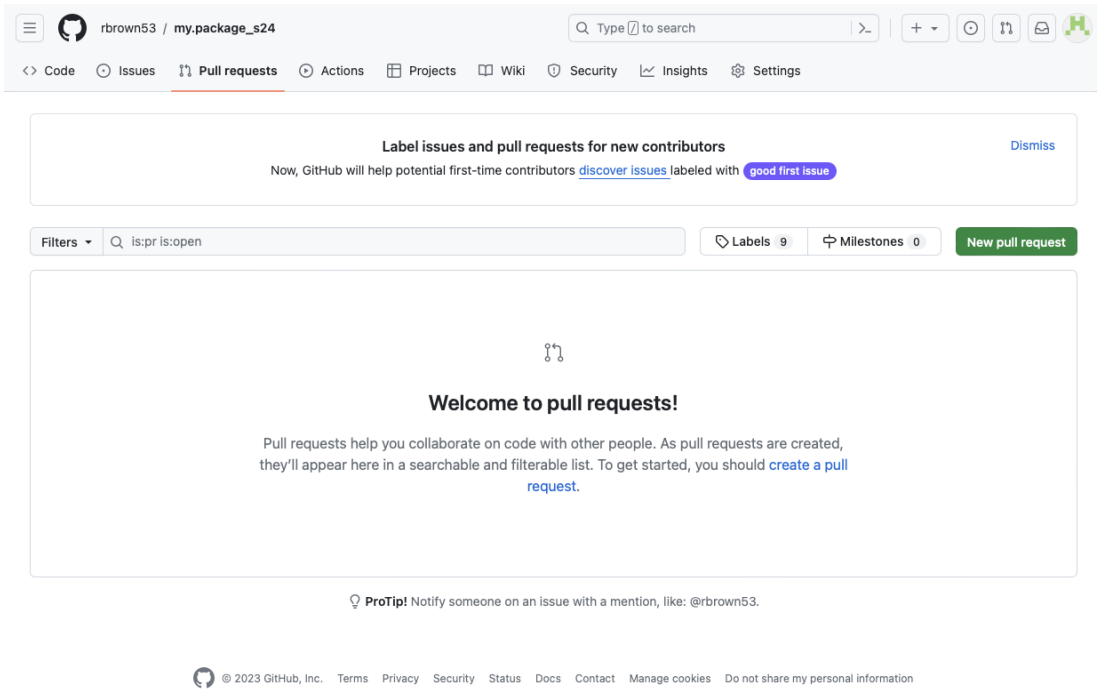
```
git init
```

3. Use the **add** and **commit** commands to add to the local repository
4. Connect the local and remote repos and push:

```
git remote add origin \
https://github.com/username/Math_3190_Assignments.git
git push -u origin main
```

Pull Requests

Pull requests enable sharing of changes from other branches/forks of a repo. Potential changes can be reviewed before they are merged into the base branch. More details on how to do these will be provided in the lab.



Extra - .gitignore File

When we initialize Git in a folder, a hidden folder called `.git` is created. Another hidden file that is useful is `.gitignore`, which lets Git know what files it should NOT keep track of. We can add this when creating a repo on GitHub or we can add this on the local machine by navigating to the directory in the terminal and typing

```
touch .gitignore
```

Extra - Hidden Files

These file and folders that begin with a period (.) are hidden by default, but sometime we want to edit them, especially the `.gitignore`. We can see hidden files by doing the following:

Mac: In Finder, type `Shift-Command-.` (period). Type that again to hide the files again.

Windows 10:

1. Open File Explorer from the taskbar.
2. Select View > Options > Change folder and search options.
3. Select the View tab and, in Advanced settings, select Show hidden files, folders, and drives and OK.

Windows 11:

1. Open File Explorer from the taskbar.
2. Select View > Show > Hidden items.

Conclusion

Positron also has a more visual version control that uses Git built in. It is on the left side and its symbol looks like a branching path.

On Canvas, you'll find a file called "Unix and Git Cheat Sheet.pdf" that contains all of the Git and Unix commands that are contained in these notes plus more.

Session info

```
sessionInfo()
```

```
R version 4.5.2 (2025-10-31)
Platform: aarch64-apple-darwin20
Running under: macOS Sequoia 15.7.2

Matrix products: default
BLAS:   /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.framework/Versions/A/libBL
LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib;  LAPACK version 3.12

locale:
[1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8

time zone: America/Denver
tzcode source: internal

attached base packages:
[1] stats     graphics  grDevices utils     datasets  methods   base

loaded via a namespace (and not attached):
 [1] compiler_4.5.2    fastmap_1.2.0     cli_3.6.5         tools_4.5.2
 [5] htmltools_0.5.9   rstudioapi_0.17.1 yaml_2.3.12       rmarkdown_2.30
 [9] knitr_1.51        jsonlite_2.0.0    xfun_0.55         digest_0.6.39
[13] rlang_1.1.6       evaluate_1.0.5
```